

System-Level I/O

Outline

- Unix I/O
- Reading File Metadata
- Directory
- Sharing Files & I/O redirection
- Suggested Reading
 - 10.1~10.4, 10.6~10.9

Why Unix I/O

- Input/Output
 - A process of copying data between main memory and external devices
- Standard I/O library
 - User level or high level I/O functions
- Unix I/O
 - I/O functions provided by kernel
 - Helps you understand other system concepts
 - I/O vs. process
 - Sometimes, it is the only choice

Unix I/O

- A Unix file is a sequence of m bytes
 - $B_0, B_1, \dots, B_k, \dots, B_m$
- All I/O devices are modeled as files
 - such as networks, disks, and terminals
 - allows Unix to export a simple, low-level application interface, known as Unix I/O
- All input and output is performed by reading and writing the appropriate files
 - Enables all input and output to be performed in a uniform and consistent way.

File Type

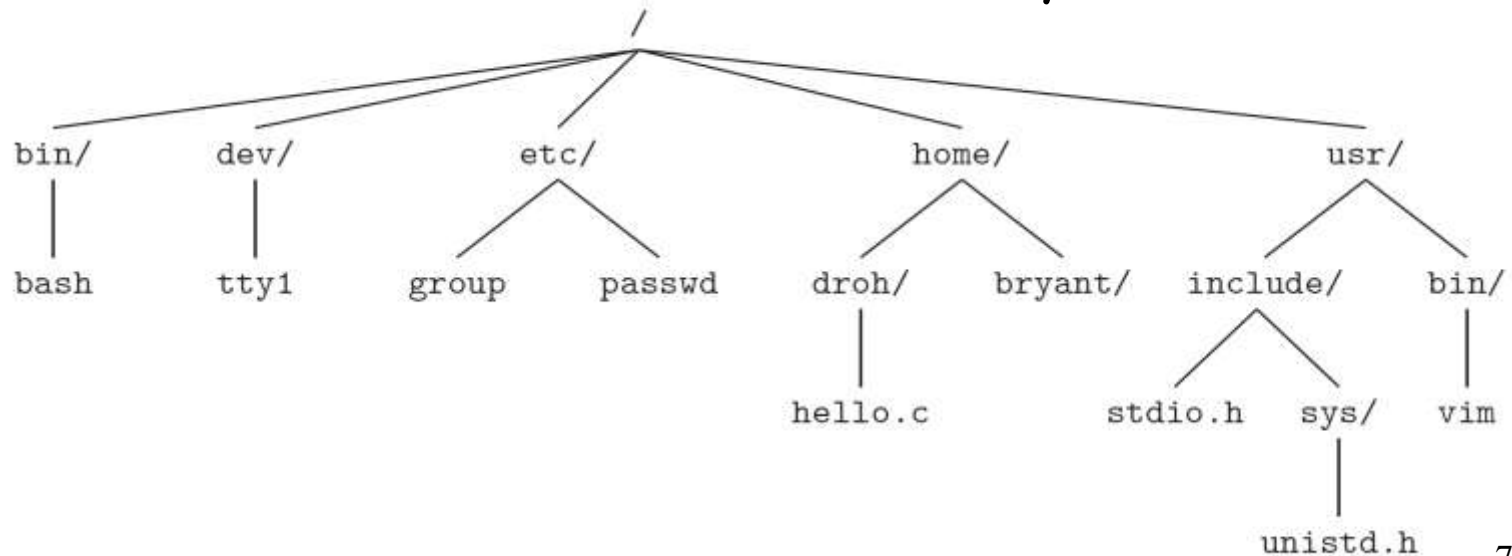
- *regular file*: contains arbitrary data
 - from the application point of view, there are two kinds of regular files
 - *text files*: contain only ASCII or Unicode characters
 - *binary files*: which are everything else
 - to the kernel there is no difference
- A Linux text file consists of
 - a sequence of *text lines*, where each line is a sequence of characters terminated by a *newline* character ('\n') which is the ASCII line feed character (LF) with value 0x0a.

File Type

- *A directory is a file consisting of*
 - an array of *links*,
 - each link maps a *filename* to a file
 - which may be another directory
 - each directory contains at least two entries:
 - . (dot) is a link to the directory itself
 - .. (dot-dot) is a link to the *parent directory*
 - related command
 - mkdir: create a directory
 - ls: view directory contents
 - rmdir: delete a directory

Directory Hierarchy

- The Linux kernel
 - organizes all files in a single *directory hierarchy*
 - anchored by the *root directory* named / (slash)
 - Each file in the system is a direct or indirect descendant of the root directory



File Type

- A *socket* is a file that is used to communicate with another process across a network
- Other file types include
 - *named pipes*,
 - *symbolic links*,
 - *character and block devices*

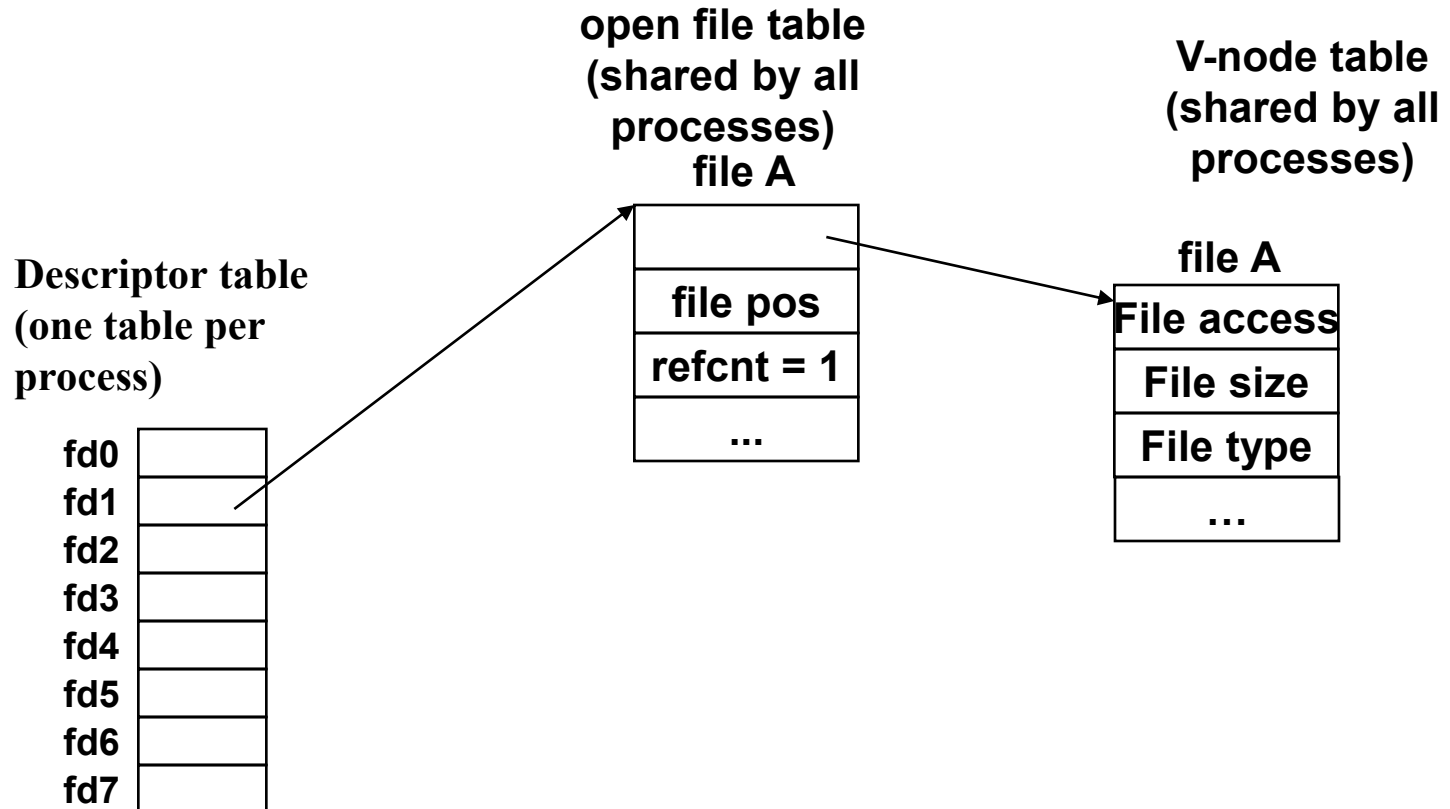
Directory Hierarchy

- Each process has a *current working directory*
 - identifies its current location in the directory hierarchy
- `cd` command
 - change the shell's current working directory
- Locations in the directory hierarchy are specified by *pathnames*
 - *absolute pathname*: `/home/droh/hello.c`
 - *relative pathname*: `./hello.c`, `../../home/droh/hello.c`
(if `/home/droh` is the current working directory)

Open Files

- An application announces its intention to access an I/O device
 - The kernel opens the corresponding file
 - The kernel returns a small non-negative integer, descriptor
 - Identifies the file in all subsequent operations on the file
 - The kernel keeps track of all information about the open file
 - Maintains a file position k , initially 0, for each open file

Kernel Data Structures for Files



Descriptor table

- Each process has its own separate descriptor table
 - whose entries are indexed by the process's open file descriptors
 - each open descriptor entry points to an entry in the file table

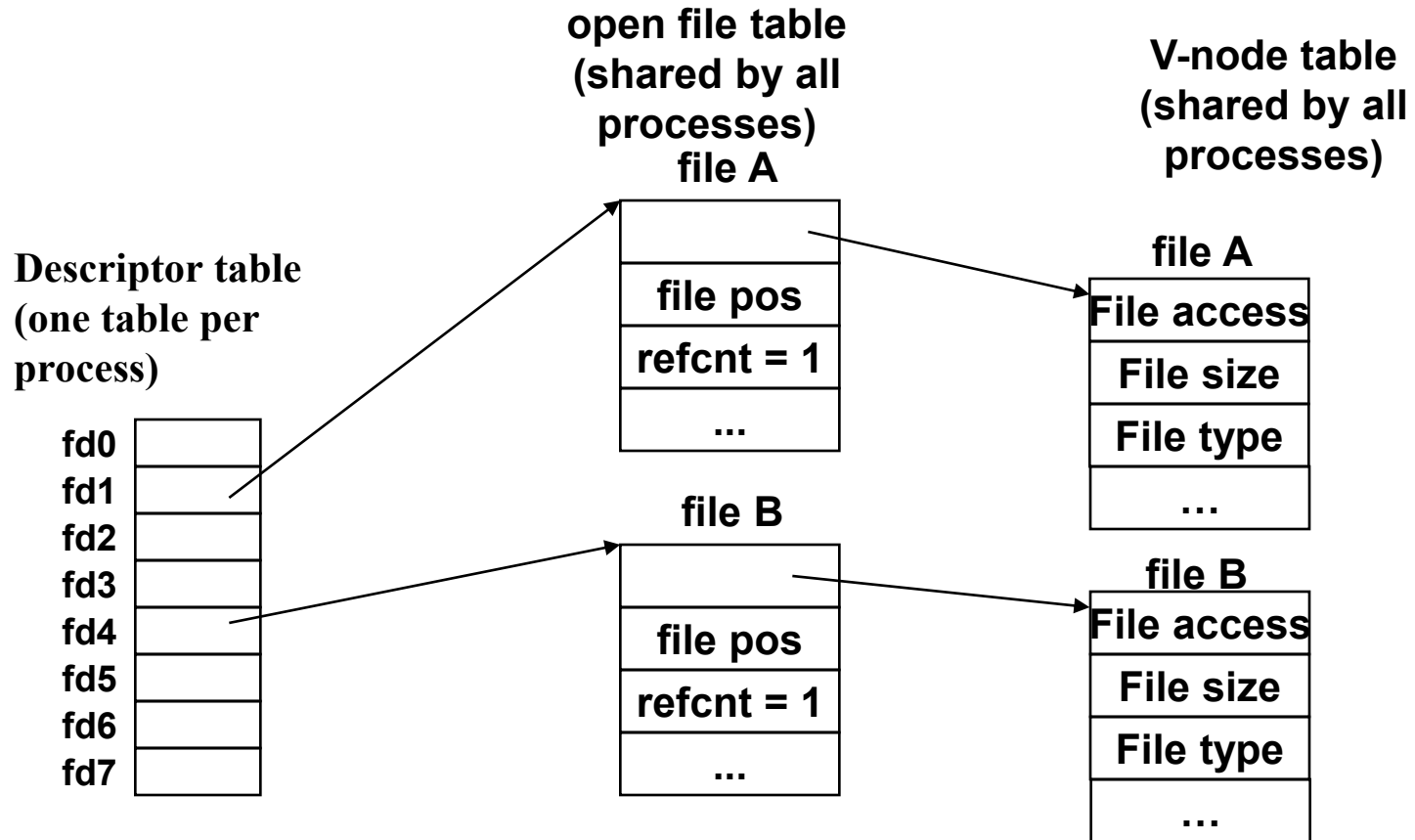
File table

- The set of open files is represented by a file table
- File table is shared by all processes
- Each file table entry consists
 - the current file position
 - a reference count of the number of descriptor entries that currently point to it
 - a pointer to an entry in the v-node table
 - the kernel deletes a file table entry when its reference count becomes zero

V-node table

- V-node table is shared by all processes
- Each entry contains most of the information of a file

Kernel Data Structures for Files



Open Files

- An application announces its intention to access an I/O device
 - The application keeps track of only the descriptor
 - An application can set the current file position explicitly by performing a **seek** operation

Open Files

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int open(char *filename, int flags, mode_t mode);
```

Returns: new file descriptor if OK, -1 on error

flags

- O_RDONLY, O_WRONLY, O_RDWR (must have one)
- O_CREAT, O_TRUNC, O_APPEND (optional)

mode

- S_IRUSR, S_IWUSR, S_IXUSR
- S_IRGRP, S_IWGRP, S_IXGRP
- S_IROTH, S_IWOTH, S_IXOTH

Open Files

`umask() : set mask of process`

```
#define DEF_MODE S_IRUSR | S_IWUSR |      \  
                S_IRGRP | S_IWGRP |      \  
                S_IROTH | S_IWOTH  
#define DEF_UMASK S_IWGRP | S_IWOTH  
  
umask(DEF_UMASK) ;  
fd = open ("foot.txt",  
           O_CREAT|O_TRUNC|O_WRONLY, DEF_MODE) ;  
/* permission: S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH */
```

Open Files

```
#include "csapp.h"
2
3 int main()
4 {
5     int fd1, fd2;
6
7     fd1 = Open("foo.txt", O_RDONLY, 0);
8     Close(fd1);
9     fd2 = Open("baz.txt", O_RDONLY, 0);
10    printf("fd2 = %d\n", fd2);
11    exit(0);
12 }
```

Close Files

- close the file
 - Do not access the file
 - The kernel action
 - The kernel frees the structures it created when the file was opened
 - The kernel restores the descriptor to a pool of available descriptors
 - The next file that is opened is guaranteed to receive the smallest available descriptor in the pool

Close Files

- close the file
 - Default actions when terminate
 - The kernel closes all open files
 - The kernel frees their memory resources

```
#include <unistd.h>  
int close(int fd) ;
```

Returns: zero if OK, -1 on error

Reading and Writing Files

- A read operation
 - Copies $m > 0$ bytes from the file to memory
 - starting at the current file position k
 - incrementing k by m
 - If the total bytes from k to the end of file is less than m , it triggers a condition known as end-of-file (EOF)
 - Can be detected by the application
 - There is no explicit "EOF character" at the end of a file

Reading and Writing Files

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count);
```

returns: number of bytes read if OK,
0 on EOF, -1 on error

```
ssize_t write(int fd, const void *buf, size_t count);
```

returns: number of bytes written if OK,
-1 on error

Reading and Writing Files

```
1  #include "csapp.h"
2
3  int main(void)
4  {
5      char c;
6
7      /* copy stdin to stdout, one byte at a time */
8      while(Read(STDIN_FILENO, &c, 1) != 0)
9          Write(STDOUT_FILENO, &c, 1);
10     exit(0);
11 }
```

STDIN_FILENO(0), STDOUT_FILENO(1), STDERR_FILENO(2)

Reading and Writing Files

- `ssize_t` vs. `size_t`
- Short count
 - Read and write transfer fewer bytes than the application requests
 - Encounter EOF on reads
 - Reading text lines from a terminal
 - Reading and writing network sockets

Reading File Metadata

```
#include <unistd.h>
```

```
#include <sys/stat.h>
```

```
int stat(const char *filename, struct stat *buf);
```

```
int fstat(int fd, struct stat *buf) ;
```

returns: 0 if OK, -1 on error

Reading File Metadata

```
/* file info returned by the stat function */
struct stat {
    dev_t st_dev;           /* device */
    ino_t st_ino;           /* inode */
    mode_t st_mode;         /* protection and file type */
    nlink_t st_nlink;       /* number of hard links */
    uid_t st_uid;           /* user ID of owner */
    gid_t st_gid;           /* group ID of owner */
    dev_t st_rdev;          /* device type (if inode device) */
    off_t st_size;          /* total size, in bytes */
    unsigned long st_blksize; /* blocksize for filesystem I/O */
    unsigned long st_blocks; /* number of blocks allocated */
    time_t st_atime;        /* time of last access */
    time_t st_mtime;        /* time of last modification */
    time_t st_ctime;        /* time of last change */
};
```

```

int main(int argc, char **argv)
{
    struct stat stat ;
    char *type, *readok ;
    Stat(argv[1], &stat) ;
    if (S_ISREG(stat.st_mode)) /* Determine file type */
        type = "regular" ;
    else if (S_ISDIR(stat.st_mode))
        type = "directory" ;
    else
        type = "other" ;

    if ((stat.mode & S_IRUSR)) /* check read access */
        readok = "yes";
    else
        readok = "no";

    printf("type: %s, read: %s\n", type, readok) ;
    exit(0) ;
}

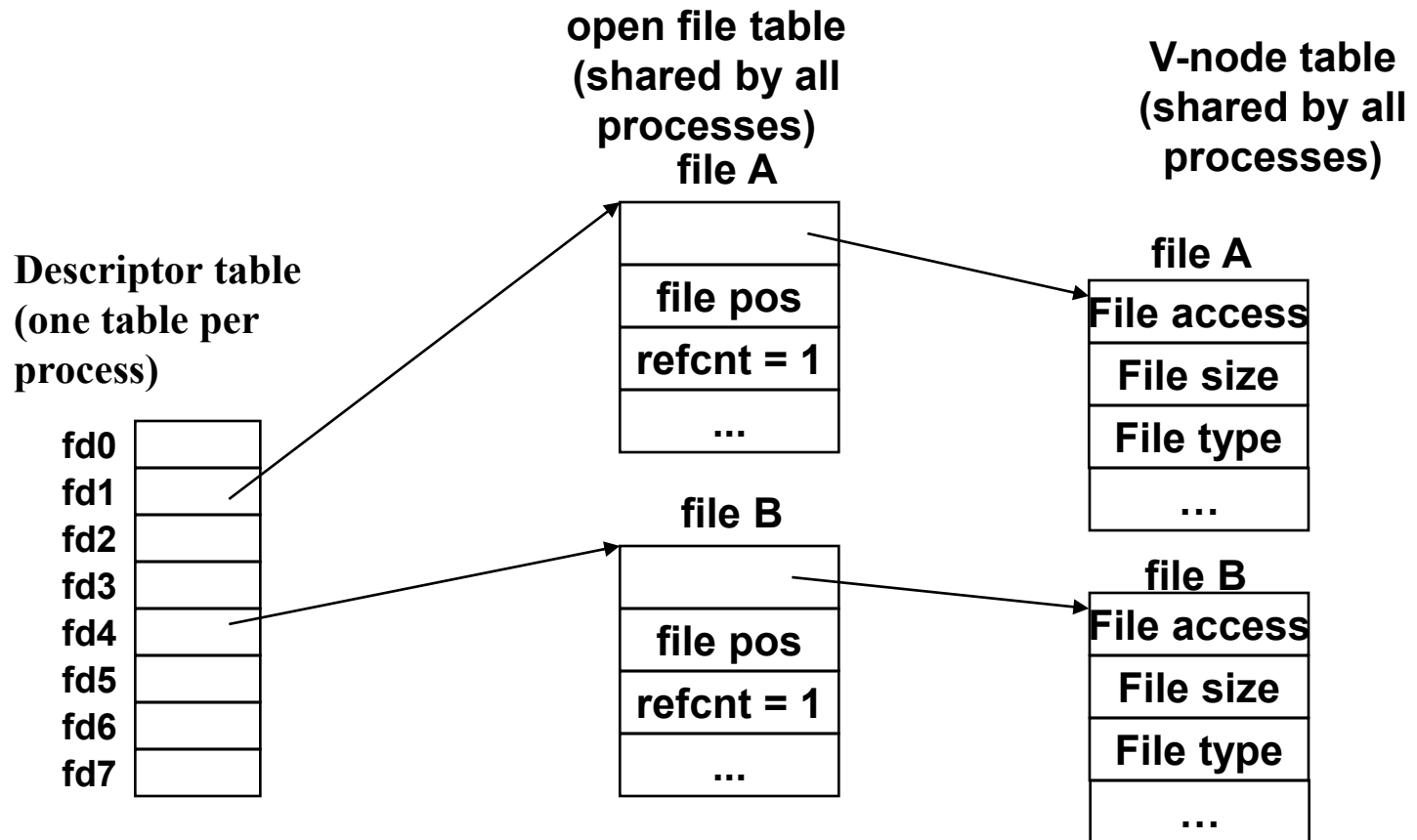
```

S_ISREG() Is this a regular file?
 S_ISDIR() Is this a directory file?

V-Node table

- Shared by all processes
- Each entry contains most of the information in the stat structure, such as
 - st_mode
 - st_size

V-Node table



Read Directory Contents

```
struct dirent {  
    ino_t d_ino; /*inode number*/  
    char d_name[256]; /*file name */  
}
```

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
DIR *opendir(const char *filename);
```

returns: pointer to handle if OK, NULL on error

```
struct dirent *readdir(DIR *dirp);
```

returns: pointer to next directory entry if OK,
NULL if no more entry or error

```
int closedir(DIR *dirp);
```

returns: 0 if OK, -1 on error

```
1. #include "csapp.h"
2.
3. int main(int argc, char **argv)
4. {
5.     DIR *stream;
6.     struct dirent *dep;
7.
8.     steamp = Opendir(argv[1]);
9.
10.    errno = 0;
11.    while ((dep = Readdir(stream)) != NULL) {
12.        printf("Found file: %s\n", dep->d_name);
13.    }
14.    if (errno != 0)
15.        unix_error("readdir error");
16.
17.    Closedir(stream);
18.    exit(0);
19. }
```


Sharing Files

```
#include "csapp.h"
```

```
int main()
```

```
{
```

```
    int fd1, fd2;
```

```
    char c;
```

```
    fd1 = open("foobar.txt", O_RDONLY, 0) ;
```

```
    fd2 = open("foobar.txt", O_RDONLY, 0) ;
```

```
    read(fd1, &c, 1) ;
```

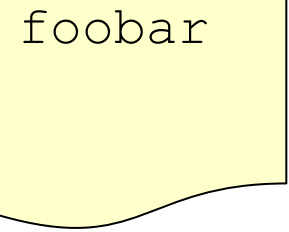
```
    read(fd2, &c, 1) ;
```

```
    printf("c = %c\n", c) ;
```

```
    exit(0)
```

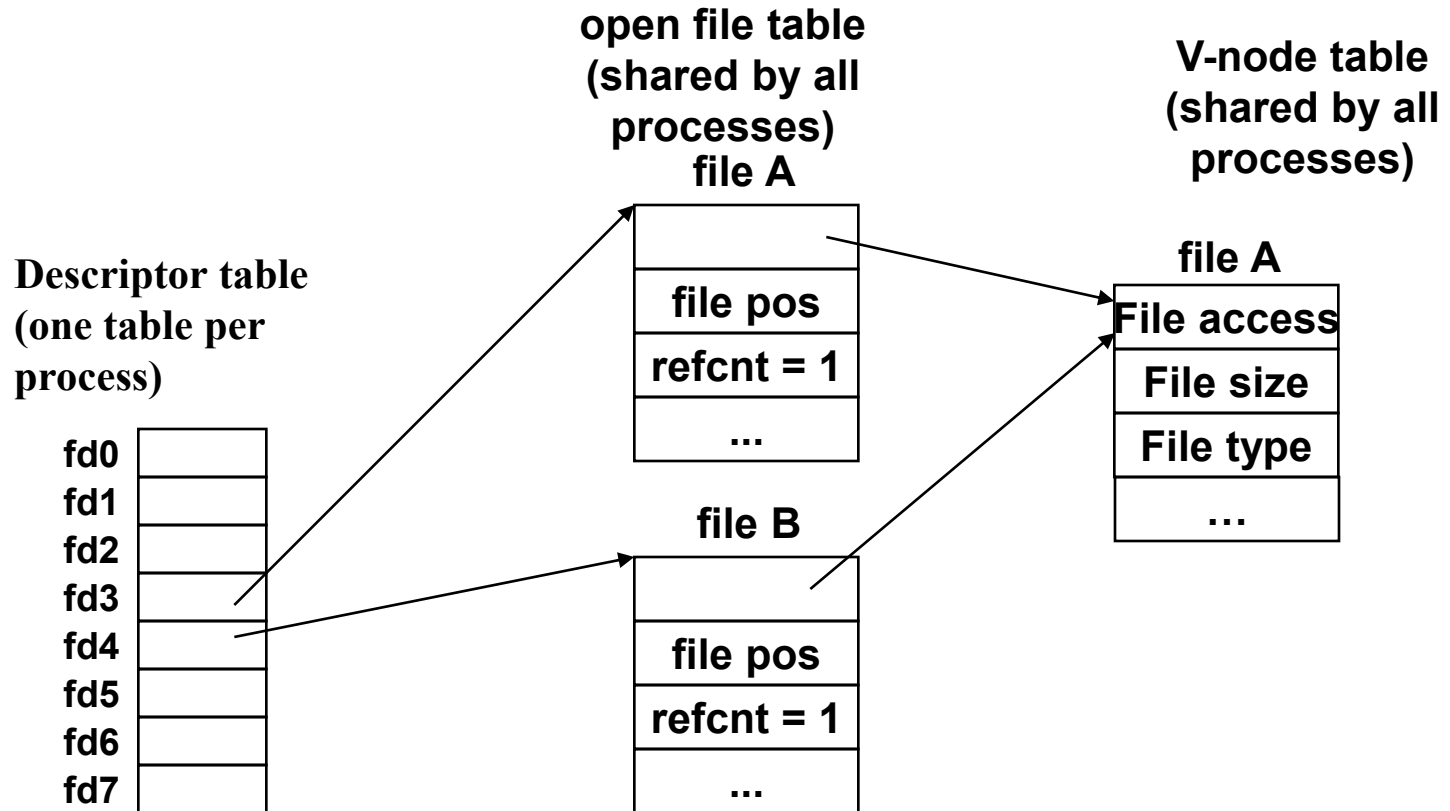
```
}
```

foobar.txt



foobar

Sharing Files



Sharing Files between Parent and Child

```
#include "csapp.h"
```

```
int main()  
{
```

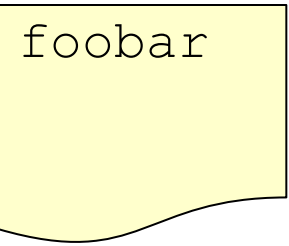
```
    int fd;  
    char c;
```

```
    fd = open("foobar.txt", O_RDONLY, 0) ;  
    if (fork() == 0) {  
        read(fd, &c, 1) ;  
        exit(0) ;  
    }
```

```
    wait(NULL) ;  
    read(fd, &c, 1) ;  
    printf("c = %c\n", c) ;  
    exit(0)
```

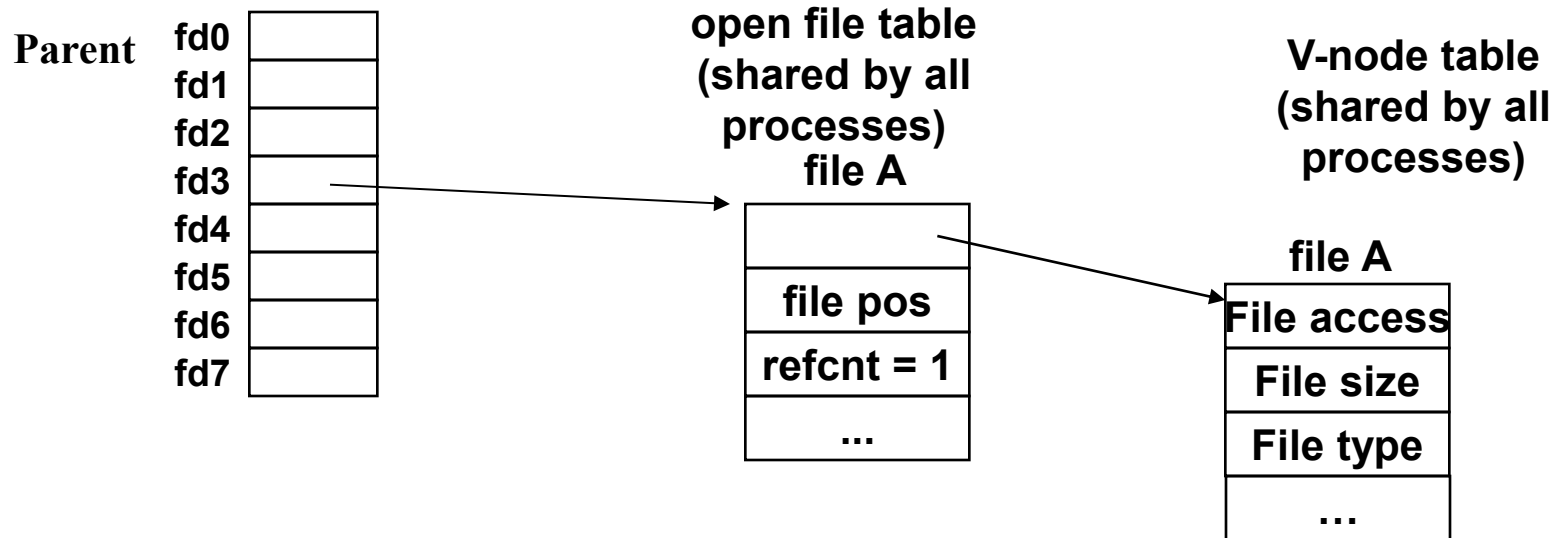
```
}
```

foobar.txt

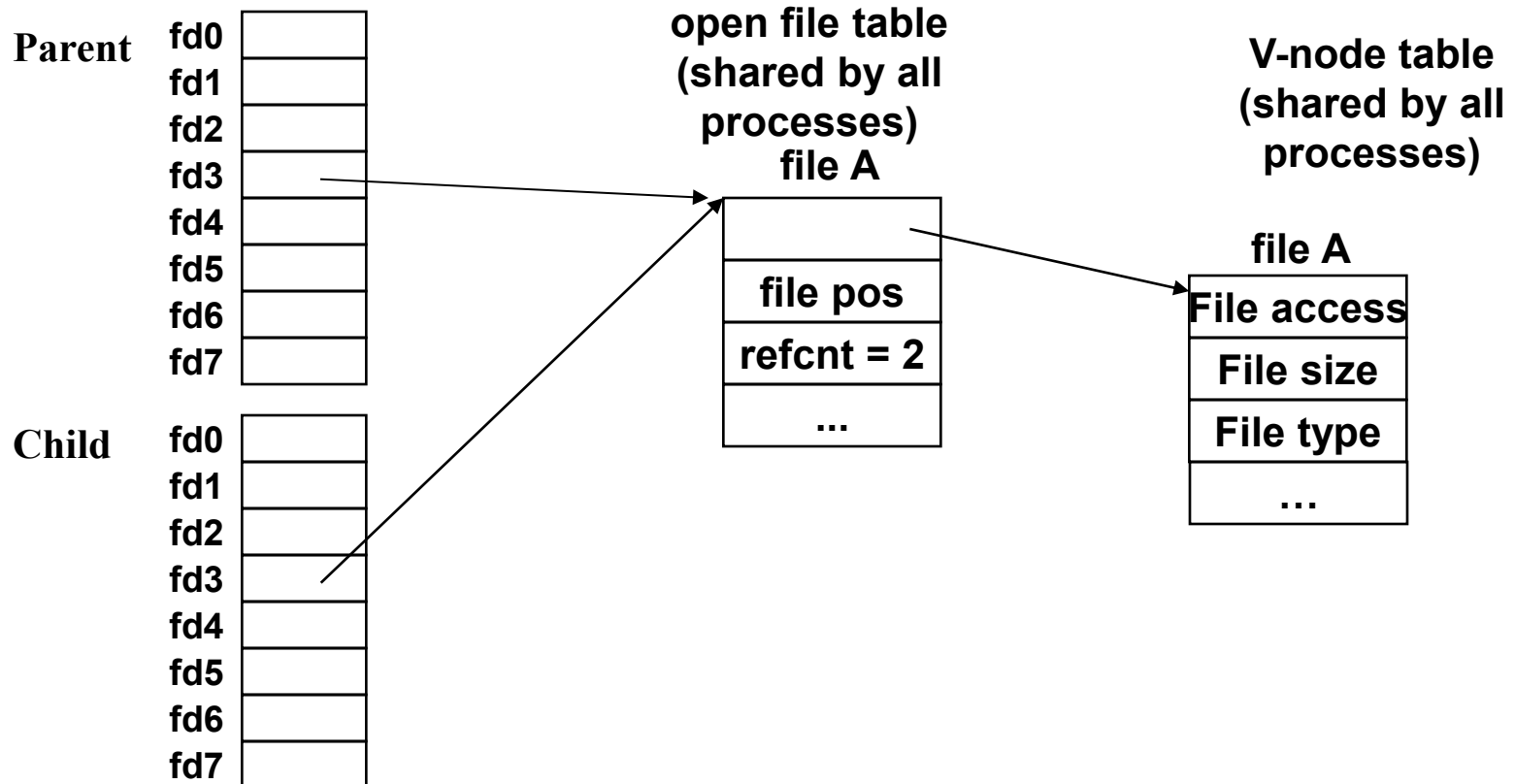


foobar

Sharing Files between Parent and Child



Sharing Files between Parent and Child



I/O Redirection

unix > `ls > foo.txt`

- `dup2` copies entries in the per-process file descriptor table.

```
#include <unistd.h>
```

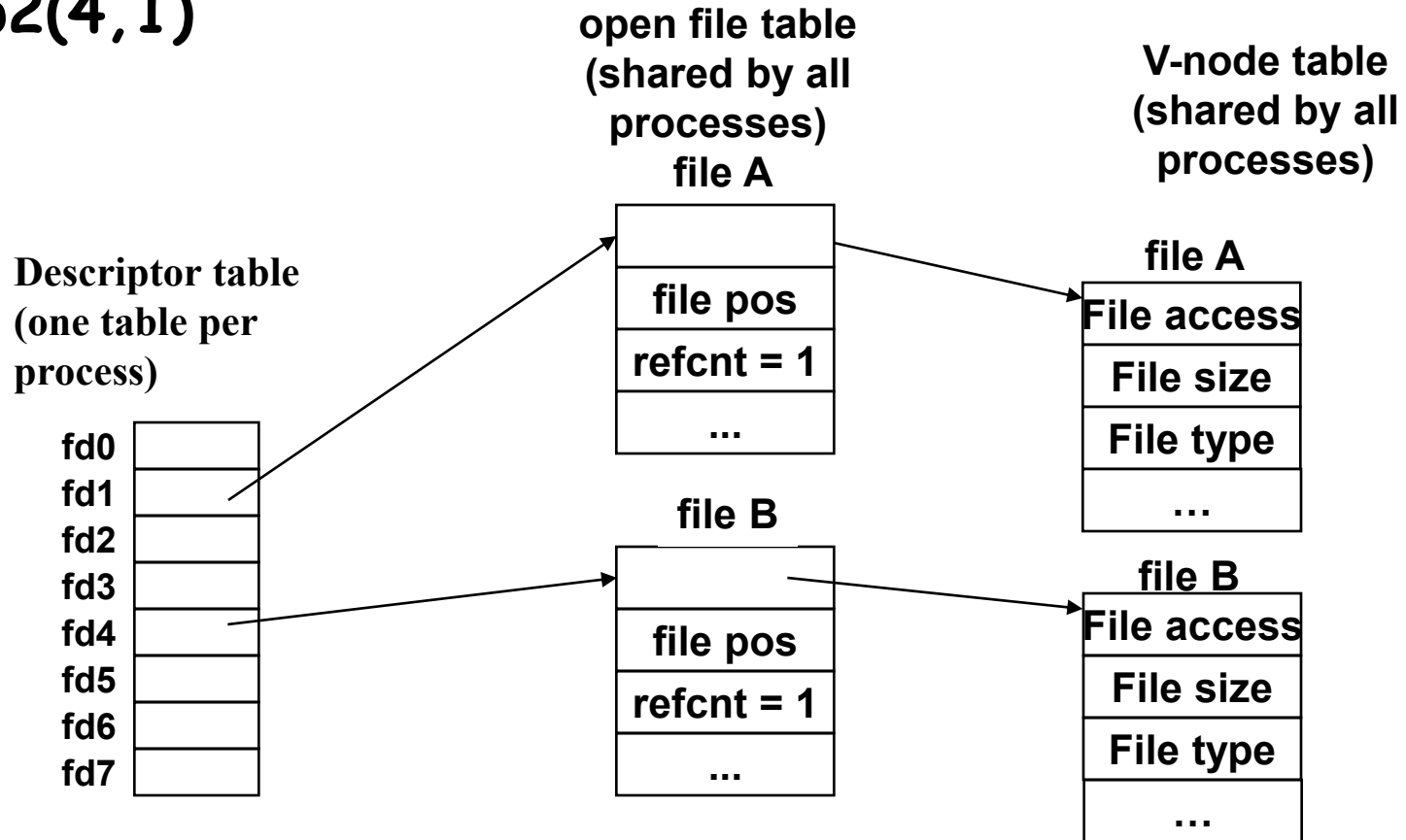
```
int dup2(int oldfd, int newfd);
```

**returns: nonnegative descriptor if OK,
-1 on error**

- `dup2(oldfd, newfd)` overwrites the entry in the per-process file table for `newfd` with the entry for `oldfd`.

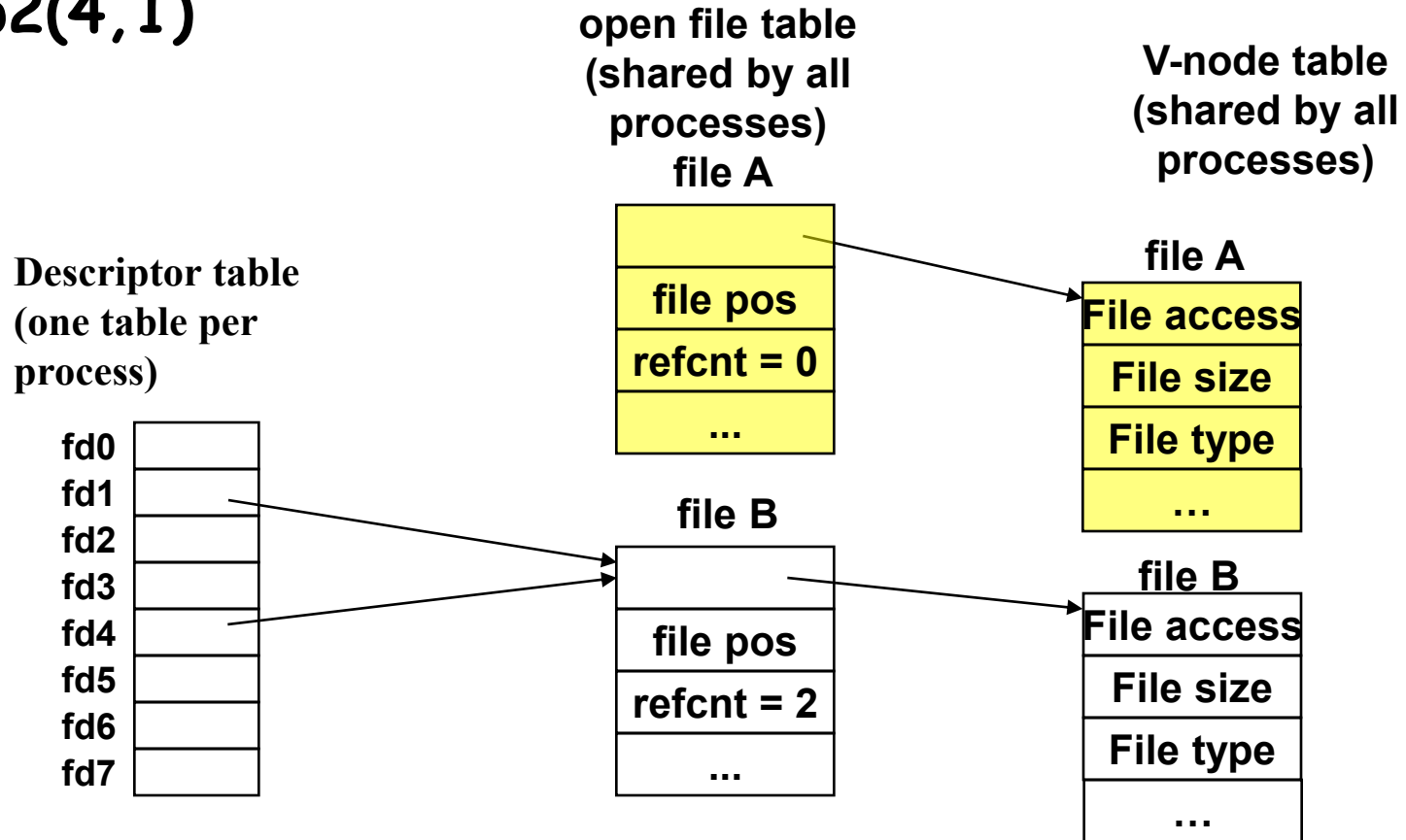
Redirection

`dup2(4,1)`



Redirection

`dup2(4,1)`



Redirection

```
#include "csapp.h"
```

```
int main()
```

```
{
```

```
    int fdA, fdB;
```

```
    char c;
```

```
    fdA = open("foobar.txt", O_RDONLY, 0) ;
```

```
    fdB = open("foobar.txt", O_RDONLY, 0) ;
```

```
    read(fdB, &c, 1) ;
```

```
    dup2(fdB, fdA) ;
```

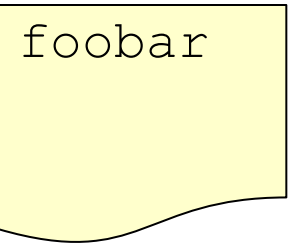
```
    read(fdA, &c, 1) ;
```

```
    printf("c = %c\n", c) ;
```

```
    exit(0)
```

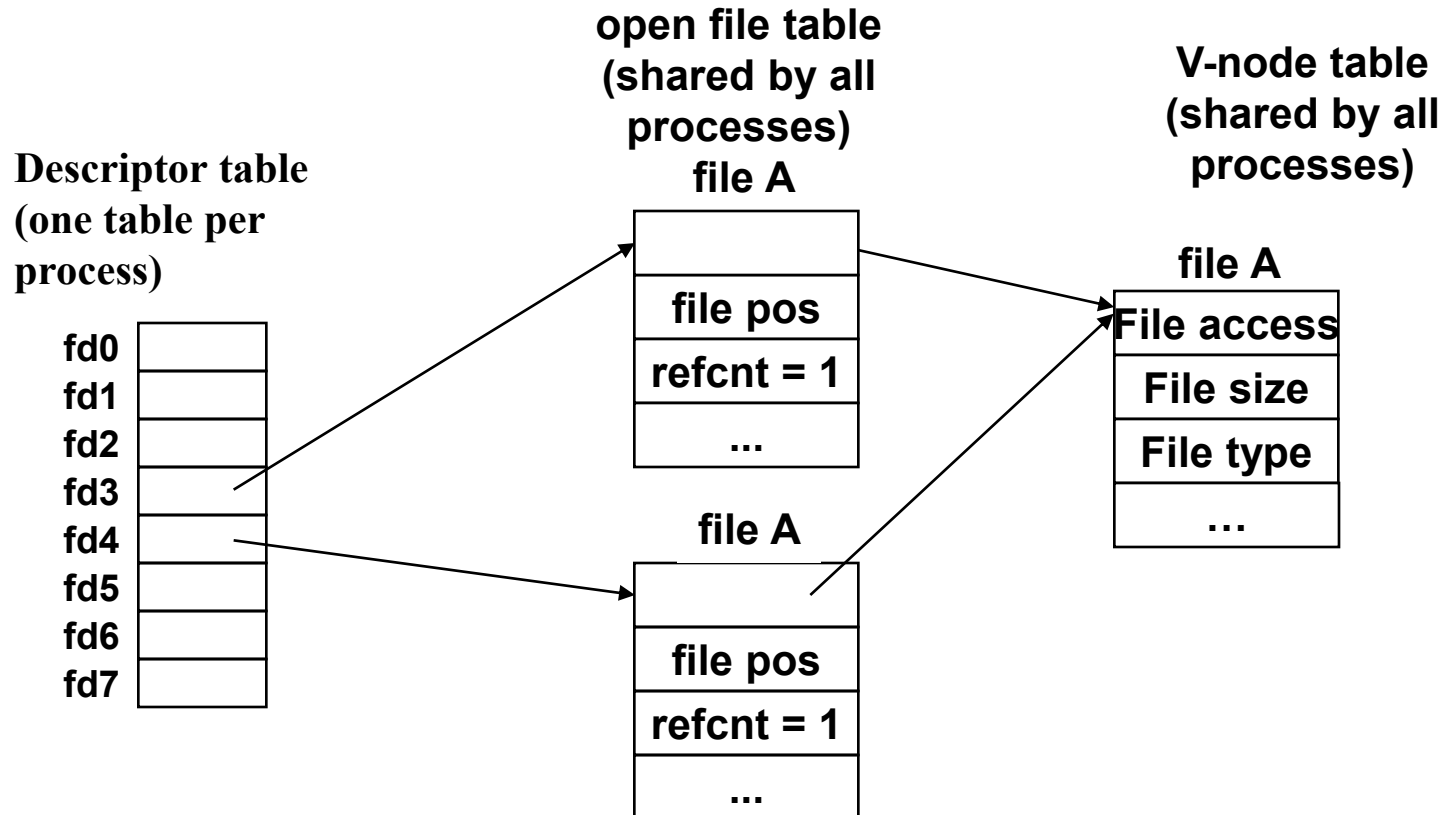
```
}
```

foobar.txt

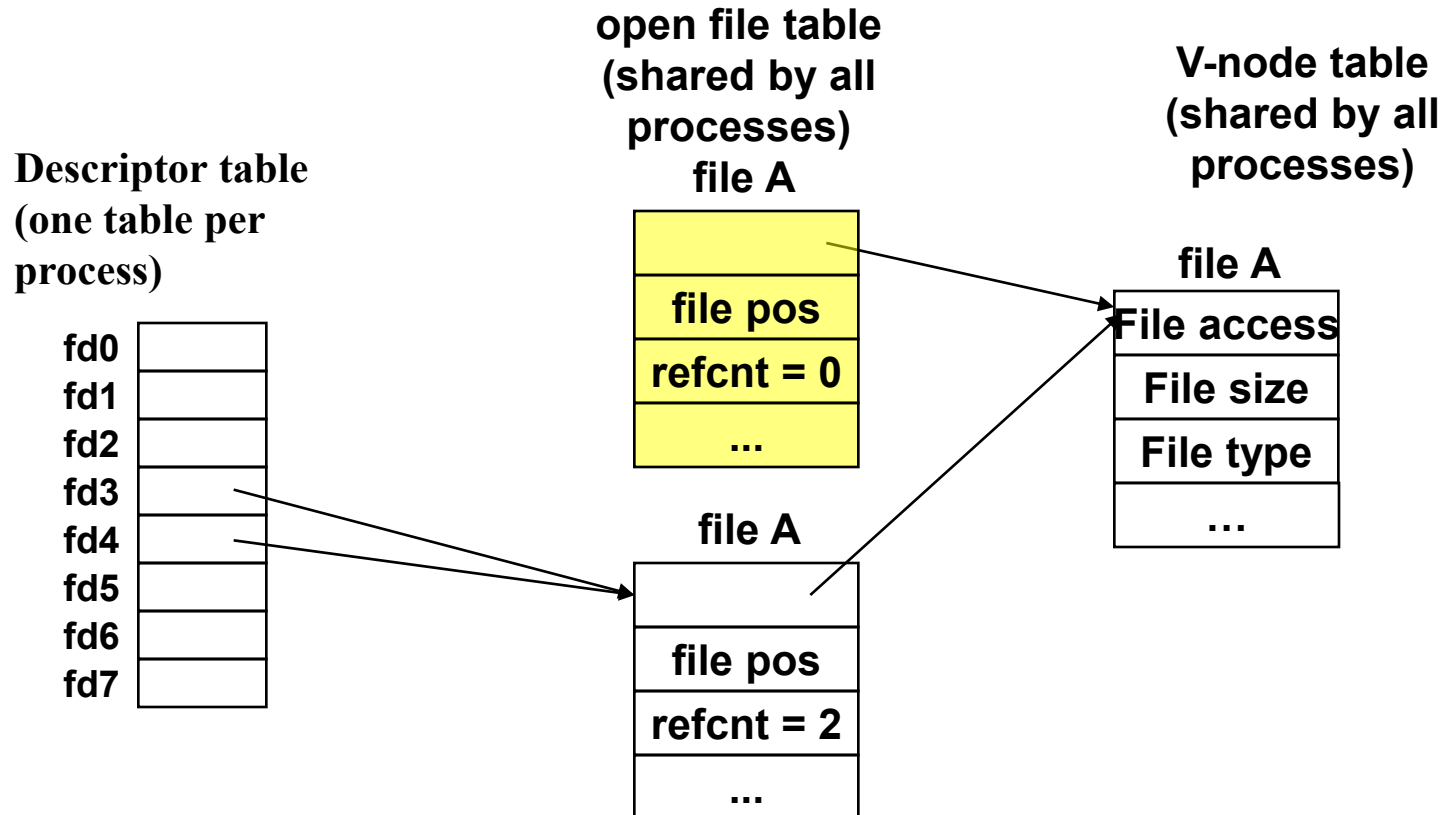


foobar

Redirection



Redirection



Question: How to impl I/O Redirection with dup2

unix > ls > foo.txt

- int dup2(int oldfd, int newfd)
 - How could shell do?
 1. fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC);
 2. dup2(output_fd, stdout); **Child inherits father's fd table**
 3. if (pid = fork() == 0) execve("ls", argv, environ);
 4. waitpid(pid, NULL, 0);
- Q: Any other issue?**

How to restore stdout in Shell

- `int dup(int oldfd)`
 - Like `dup2`, `dup` also duplicates `oldfd`
 - While the OS picks and returns a `newfd` for the duplication (typically the smallest usable one)
- How could shell do?
 - Before `dup2(fd, stdout)`: `save_stdout = dup(stdout);`
 - After fork: `dup2(save_stdout, stdout);`
`close(save_stdout);`

SideBar

- dup/dup2 takes fd (int) as arguments
- stdin/stdout/stderr
 - What are their values?
 - Type: FILE *

```
printf("stdin val: 0x%lx\n", stdin);
```
- int fileno(FILE *stream);
 - fileno(stdout) is 1

Recap: Fork + Print + Dup2

```
tmac@intel12-pc:~/ics/exception-process$ cat fork.c
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    for (int i = 0; i < 2; ++i) {
```

```
        fork();
```

```
        printf("Hello ICS\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
tmac@intel12-pc:~/ics/exception-process$ ./a.out
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

```
Hello ICS
```

Recap: Fork + Print + Dup2

```
tmac@intel12-pc:~/ics/exception-process$ ./a.out
Hello ICS
Hello ICS
Hello ICS
Hello ICS
Hello ICS
Hello ICS
```

```
tmac@intel12-pc:~/ics/exception-process$ ./a.out > tmp
tmac@intel12-pc:~/ics/exception-process$ cat tmp
Hello ICS
Hello ICS
Hello ICS
Hello ICS
Hello ICS
Hello ICS
Hello ICS
Hello ICS
```


How to fix the print times

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 int main()
6 {
7     for (int i = 0; i < 2; ++i) {
8         fork();
9         printf("Hello ICS\n");
10        fflush(stdout);
11    }
12    return 0;
13 }
```